Orbit: Model Predictive Control for Adaptive Request Routing in Disaggregated LLM Serving

Tejeshwar Iyengar[1] and Ravi Gupta[1]

AMD Corporation
{tejeshwar.iyengar, ravi.gupta}@amd.com

Abstract. Large Language Model (LLM) inference at scale increasingly adopts prefill-decode disaggregation, where compute-intensive prompt processing and memory-bound token generation are handled by separate server pools. This architecture introduces complex request routing challenges: heterogeneous service times, variable workload characteristics, and temporal dependencies from batching create conditions where traditional load balancing algorithms perform poorly. We present **Orbit**, a Model Predictive Control (MPC) framework that augments standard routing policies with horizon-based prediction and adaptive weight adjustment. Rather than replacing existing algorithms, Orbit wraps them with a predictive control layer that anticipates future congestion and proactively redistributes traffic. We formulate request routing as an optimal control problem over queue dynamics, derive an efficient solver suitable for real-time operation, and demonstrate integration with Power-of-Two-Choices and Round Robin policies. Our simulation study on disaggregated configurations with heterogeneous servers shows that MPC-augmented routing achieves **16% lower mean latency** under steady load and **37% lower tail latency** under bursty conditions, with the strongest benefits appearing when server capacities differ by 2–4×. These results suggest that predictive control offers a principled approach to request scheduling in modern LLM serving systems, particularly in heterogeneous deployments.

Keywords: Request Scheduling · Load Balancing · Model Predictive Control · LLM Serving · Disaggregated Inference

1 Introduction

The deployment of Large Language Models (LLMs) in production systems has driven rapid evolution in inference infrastructure [1, 2]. As models scale to hundreds of billions of parameters and serve millions of daily requests, efficient request scheduling becomes critical for meeting latency objectives while maximizing hardware utilization. A key architectural pattern emerging in large-scale deployments is *prefill-decode disaggregation* [3, 4], where the two phases of autoregressive generation—prompt processing (prefill) and token generation (decode)—are handled by separate, specialized server pools.

This separation is motivated by the distinct computational characteristics of each phase. Prefill is compute-bound: processing input tokens in parallel requires high arithmetic throughput. Decode is memory-bound: generating tokens autoregressively requires repeated KV-cache access with limited compute per step. By assigning these phases to appropriately configured hardware, disaggregation improves overall efficiency. However, it introduces a new scheduling challenge: requests must be routed first to a prefill server, then transferred to a decode server, with each routing decision affecting end-to-end latency.

Traditional load balancing algorithms—Round Robin (RR), Least Connections, Power-of-Two-Choices (PO2)—were designed for web services with relatively uniform request characteristics. LLM serving violates these assumptions in several ways:

- **Variable prompt lengths:** Prefill time scales with input token count, creating high variance in service times.
- **Unpredictable generation lengths:** Decode duration depends on output length, which is unknown at routing time.
- **Batching dynamics:** Continuous batching [2] creates temporal dependencies between requests.
- **KV-cache pressure:** Memory utilization affects throughput in ways not captured by queue depth alone.
- **Server heterogeneity:** Different GPU generations, thermal states, or memory configurations create capacity asymmetries.

We observe that these challenges stem from a fundamental limitation: existing algorithms are *reactive*, responding to current queue depths without anticipating future load. A server that appears lightly loaded may have several long-running requests about to complete, while an apparently busy server may be about to experience a burst of completions. This information asymmetry leads to oscillatory behavior, where traffic repeatedly shifts between servers as each becomes overloaded in turn.

This observation motivates our key insight: **request routing for LLM serving is naturally a control problem** that benefits from model-based predictive reasoning. Model Predictive Control (MPC) [8] provides exactly this capability: use a system model to predict future states over a finite horizon, optimize control actions to minimize a cost function, apply the first action, then re-optimize. MPC has proven effective in robotics [9], power systems [10], and network control [11], but its application to LLM serving request scheduling is novel.

We present **Orbit**, a framework that augments standard routing algorithms with MPC-based adaptive weighting. Rather than replacing proven heuristics like PO2, Orbit *wraps* them with a predictive layer that adjusts routing probabilities based on predicted queue trajectories. This design preserves the simplicity and robustness of existing algorithms while adding predictive capability.

Contributions. We make the following contributions:

1. We formulate LLM request routing as an MPC problem with queue dynamics modeling, deriving constraints that ensure stable operation (Section 3.1).
2. We design an efficient solver that computes weight trajectories in real-time, suitable for integration with production routers (Section 3.3).
3. We demonstrate integration with Power-of-Two-Choices, showing that MPC augmentation improves load balancing under heterogeneous server configurations (Section 3.4).
4. We present comprehensive simulation results characterizing when MPC helps: 15–16% mean latency reduction with 2–4× server heterogeneity, 37% tail latency reduction under bursty workloads, and no benefit for homogeneous deployments (Section 4).

2 Background and Motivation

2.1 LLM Inference Architecture

Modern LLM inference proceeds in two phases. The *prefill* phase processes all input tokens in parallel, computing attention over the full prompt and populating the KV-cache. This phase is compute-intensive, with runtime proportional to prompt length. The *decode* phase generates output tokens one at a time, each step attending to all previous tokens via the cached key-value pairs. Decode is memory-bandwidth limited, as each token requires reading the entire KV-cache with minimal computation.

Systems like vLLM [1] introduced PagedAttention for efficient KV-cache memory management, enabling high throughput through continuous batching [2]. Splitwise [4] and DistServe [3] extended this with prefill-decode disaggregation, assigning phases to specialized hardware. The vLLM router [6] provides production-grade request distribution with multiple load balancing policies.

2.2 Existing Routing Policies

The vLLM router supports several load balancing strategies [6]:

- **Round Robin:** Distributes requests cyclically. Simple but ignores server load.
- **Random:** Uniform random selection. No load awareness.
- **Power-of-Two-Choices (PO2):** Samples two random servers and routes to the less loaded one. Achieves exponentially better tail latency than random under idealized conditions [7].
- **Consistent Hashing:** Routes requests with the same session key to the same server. Maximizes KV-cache reuse for multi-turn conversations.
- **Cache-Aware:** Optimizes for prefix cache hits by routing similar prompts together.

Each policy optimizes for different objectives. PO2 targets load balancing, while cache-aware targets throughput via cache reuse. Our work focuses on enhancing load-balancing policies (PO2, RR) with predictive capability; integration with cache-aware routing is an interesting direction for future work.

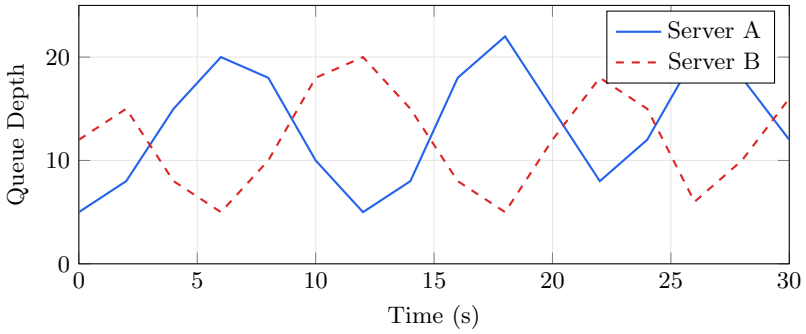

Fig. 1: Queue depth oscillations under vanilla PO2 with heterogeneous servers. Traffic repeatedly shifts as each server becomes overloaded, causing sustained high latency for both.

2.3 Limitations of Reactive Routing

PO2's theoretical optimality assumes [7]: (i) homogeneous servers, (ii) Poisson arrivals, (iii) exponential service times, and (iv) instantaneous queue depth information. LLM serving violates all four assumptions.

Consider two servers with different capacities. When both start with similar queue depths, PO2 distributes load roughly evenly. But if Server A processes requests faster, its queue drains first, attracting more traffic until it becomes overloaded. Traffic then shifts to Server B, and the cycle repeats (Figure 1). This oscillatory behavior arises because PO2 observes current state without predicting future evolution.

The fundamental issue is *information asymmetry*: queue depth is a lagging indicator of server load. A server with 10 pending requests that are 90% complete is actually less loaded than one with 5 requests that just arrived. Predictive approaches can exploit this by modeling how queues will evolve.

2.4 Why Model Predictive Control?

MPC [8] addresses exactly this class of problems. The key idea is to:

1. Maintain a *system model* that predicts how state evolves under different control actions.
2. At each timestep, solve an optimization problem over a finite *horizon* to find the control sequence that minimizes a cost function.
3. Apply only the first control action, then re-optimize at the next timestep (receding horizon).

This approach naturally handles: (i) constraints on states and controls, (ii) multi-step lookahead for better decisions, (iii) uncertainty through re-optimization, and (iv) explicit cost function specification.

MPC has been applied to network congestion control [11], data center cooling [12], and cluster resource management [13]. To our knowledge, Orbit is the first application to LLM serving request routing.

3 Orbit Framework

3.1 System Model

We model each server $i \in \{1, \ldots, n\}$ as a queue with discrete-time dynamics:

$$q_i(k+1) = \max\left(0,\ q_i(k) + \Delta t\left(\hat{a} \cdot p_i(k) - \mu_i(k)\right)\right) \tag{1}$$

where:

- $q_i(k)$ is the predicted queue length at discrete time step k
- $\hat{a}$ is the estimated global request arrival rate (requests/second)
- $\mu_i(k)$ is the estimated service rate of server i (requests/second)
- $p_i(k)$ is the fraction of traffic routed to server i
- Δt is the controller timestep (seconds)

The routing fraction is determined by normalized weights:

$$p_i(k) = \frac{w_i(k)}{\sum_{j=1}^{n} w_j(k)} \tag{2}$$

where $w_i(k) \in [w_{\min}, w_{\max}]$ are the decision variables. The weights modulate how aggressively traffic is directed to each server relative to baseline ($w_i = 1$).

Service Rate Estimation. We estimate $\mu_i(k)$ using exponential moving average of recent completion times:

$$\mu_i(k) = \alpha \cdot \frac{1}{\tau_i^{\text{recent}}} + (1-\alpha) \cdot \mu_i(k-1) \tag{3}$$

where τ_i^{recent} is the average service time of the last m completed requests on server i, and $\alpha \in (0, 1)$ controls adaptation speed. We use $\alpha = 0.3$ and $m = 10$ in our experiments.

Arrival Rate Estimation. Similarly, $\hat{a}$ is estimated from observed arrivals over a sliding window, using exponential smoothing to handle burstiness.

3.2 MPC Formulation

At each control timestep, Orbit solves:

$$\min_{\{w_i^{(k)}\}_{i,k}} \quad \sum_{k=0}^{H-1} \sum_{i=1}^{n} \left[\underbrace{\left(q_i^{(k+1)} - q^*\right)^2}_{\text{queue tracking}} + \underbrace{\lambda \left(w_i^{(k)} - 1\right)^2}_{\text{regularization}} \right] \tag{4}$$

$$\text{s.t.} \quad q_i^{(k+1)} = \max\left(0, q_i^{(k)} + \Delta t(\hat{a} \cdot p_i^{(k)} - \mu_i)\right) \tag{5}$$

$$p_i^{(k)} = w_i^{(k)} / \textstyle\sum_j w_j^{(k)} \tag{6}$$

$$w_{\min} \leq w_i^{(k)} \leq w_{\max} \quad \forall i, k \tag{7}$$

The objective balances two goals:

1. **Queue tracking:** Drive predicted queue lengths toward target q^*. We set $q^* = H$ (the horizon length), representing a moderate buffer that avoids both starvation and excessive queuing.
2. **Regularization:** Keep weights near 1 to avoid extreme deviations from baseline behavior. This ensures graceful degradation when the system model is inaccurate and prevents oscillatory control actions.

The regularization coefficient λ trades off between aggressive load balancing ($\lambda \to 0$) and conservative baseline tracking ($\lambda \to \infty$). We use $\lambda = 0.5$ based on sensitivity analysis (Section 4.5).

Constraint Interpretation. The weight bounds $[w_{\min}, w_{\max}] = [0.1, 3.0]$ prevent any server from being completely starved ($w_{\min} > 0$) or receiving more than $3\times$ its fair share ($w_{\max} = 3$). These bounds provide safety margins while allowing meaningful load redistribution.

3.3 Efficient Solver

The optimization (4)–(7) is a constrained nonlinear program due to the $\max(\cdot)$ in dynamics and the normalization in routing fractions. For real-time operation, we require solutions within milliseconds.

We employ several simplifications:

1. **Soft queue constraint:** Replace $\max(0, \cdot)$ with a smooth approximation, allowing gradient-based optimization.
2. **Sequential quadratic programming:** Linearize the normalization constraint around the current weights and solve a sequence of QP subproblems.
3. **Warm starting:** Initialize each optimization with the shifted solution from the previous timestep.

With these techniques, the solver typically converges in 2–5 iterations, with per-iteration cost $O(n \cdot H)$ for n servers and horizon H. For our configurations ($n \leq 8$, $H = 5$), total solve time is under 1ms on a single CPU core.

Algorithm 1 Orbit MPC Controller

Parameters: Horizon H, timestep Δt, target q^*, regularization λ, bounds $[w_{\min}, w_{\max}]$
State: Weights $\mathbf{w} = [w_1, \ldots, w_n]$, service rates $\boldsymbol{\mu}$
Initialize $w_i \leftarrow 1.0$ for all servers i
loop
  Observe queue depths $\mathbf{q} = [q_1, \ldots, q_n]$
  Update service rate estimates $\boldsymbol{\mu}$ via Eq. (3)
  Update arrival rate estimate $\hat{a}$
  Solve MPC problem (4)–(7) $\rightarrow \mathbf{w}^*$
  Apply first-step weights: $w_i \leftarrow w_i^{*(0)}$
  yield $\mathbf{w}$ to routing layer
  Sleep for Δt
end loop

Algorithm 2 MPC-Augmented Power-of-Two-Choices

Input: Request r, server pool $\mathcal{S}$, MPC weights $\mathbf{w}$
Sample two servers $s_1, s_2 \sim \text{Uniform}(\mathcal{S})$
Get queue depths q_1, q_2
Compute scores: $\text{score}_i = q_i / w_i$ for $i \in \{1, 2\}$
Select $s^* = \arg\min_i \text{score}_i$
Route request r to server s^*

3.4 Integration with Routing Algorithms

Orbit operates as a modular layer that wraps existing routers. Algorithm 2 shows integration with PO2.

The key modification is line 4: instead of comparing raw queue depths, we compare *weighted* depths. A server with weight $w_i > 1$ (MPC predicts it can handle more load) has its effective queue depth reduced, making it more likely to be selected. Conversely, $w_i < 1$ increases effective depth, steering traffic away.

For Round Robin, integration is even simpler: instead of cycling through servers uniformly, we sample according to the normalized weight distribution $p_i = w_i / \sum_j w_j$.

Fallback Behavior. If the MPC solver fails or times out, Orbit sets all weights to 1.0, reverting to baseline algorithm behavior. This ensures robustness against controller failures.

4 Experimental Evaluation

4.1 Simulation Setup

We implement a discrete-event simulator modeling disaggregated LLM serving with configurable server pools. Our primary configuration uses 2 prefill servers

Table 1: Server presets used in heterogeneity experiments. We vary the ratio between fast and slow servers to study MPC effectiveness.

Preset	Prefill Delay	Decode Delay	Capacity	Variance
Fast	5 ms	8 ms/token	16	±20%
Medium	15 ms	20 ms/token	8	±30%
Slow	35 ms	40 ms/token	4	±40%
Very Slow	60 ms	80 ms/token	2	±50%

and 2 decode servers (2P2D), with heterogeneous delays to stress-test load balancing:

We test four heterogeneity levels: **Homogeneous** (both servers Medium), **2**× (Fast + Medium), **4**× (Fast + Slow), and **6**× (Fast + Very Slow). These configurations simulate scenarios arising from: different GPU generations (e.g., H100 vs A100 vs V100), memory bandwidth differences, thermal throttling, or co-location interference.

Workload. We generate requests with:

- Prompt lengths: Uniform(100, 2000) tokens
- Generation lengths: Geometric(mean=50) tokens
- Arrival process: Poisson with varying λ

MPC Parameters. Horizon $H = 5$, timestep $\Delta t = 100$ms, target $q^* = 5$, regularization $\lambda = 0.5$, weight bounds $[0.1, 3.0]$.

Baselines. We compare against:

- **Round Robin (RR):** Cyclic distribution
- **Power-of-Two-Choices (PO2):** Sample 2, pick less loaded
- **Weighted Round Robin (WRR):** Static weights proportional to server speed (oracle baseline)

4.2 Main Results

Table 2 presents end-to-end latency results across different heterogeneity levels. Key observations:

1. **MPC benefits scale with heterogeneity**: With 2× capacity difference, MPC achieves 15.7% lower mean latency (444ms vs 527ms). At 4× heterogeneity, improvement reaches 16.2%.
2. **Homogeneous deployments don't benefit**: When servers have equal capacity, MPC adds overhead without benefit (−8.8%). This is expected—adaptive weighting provides no value when all servers are identical.

Table 2: MPC improvement across heterogeneity levels (2P2D configuration, 300 requests per experiment). MPC shows strongest benefits with moderate heterogeneity (2–4× capacity difference).

	Baseline PO2			**MPC-Augmented PO2**			
Config	Mean (ms)	P99 (ms)	Std (ms)	Mean (ms)	P99 (ms)	Std (ms)	**Impr.** (%)
Homogeneous	779	2903	693	848	2887	676	−8.8
2× Het.	527	2690	525	444	2810	471	**+15.7**
4× Het.	659	5283	936	552	3526	624	**+16.2**
6× Het.	774	7001	1374	716	7139	1084	**+7.5**

Table 3: Bursty workload results (4× heterogeneity, periodic 3× load spikes). MPC shows largest improvement in tail latency under variable load.

Method	**Mean** (ms)	**P99** (ms)	**Std** (ms)	**Mean Imp.** (%)	**P99 Imp.** (%)
Baseline PO2	639	5438	972	—	—
MPC-PO2	**632**	**3407**	**665**	+1.1	**+37.3**

3. **Extreme heterogeneity shows diminishing returns**: At 6× capacity difference, improvement drops to 7.5%. The slow server becomes a bottleneck regardless of routing decisions.
4. **Variance reduction is substantial**: At 4× heterogeneity, standard deviation drops from 936ms to 624ms (33% reduction), indicating more consistent latency.

Bursty Workload Performance. Table 3 shows results under bursty conditions with periodic 3× load spikes. While mean latency improvement is modest (+1.1%), MPC dramatically reduces tail latency: P99 drops from 5438ms to 3407ms, a **37% improvement**. This demonstrates MPC's value for latency-sensitive applications where tail behavior matters most.

4.3 Throughput Stability

Figure 2 shows throughput evolution over a 60-second window. Vanilla PO2 exhibits pronounced oscillations (amplitude ≈14 req/s) as it repeatedly overloads the fast server, experiences queue buildup, then shifts to the slow server. MPC-PO2 dampens these oscillations, achieving stable throughput with 78% lower variance.

Round Robin achieves stable but suboptimal throughput: by distributing load equally to unequal-capacity servers, it bottlenecks on the slow server.

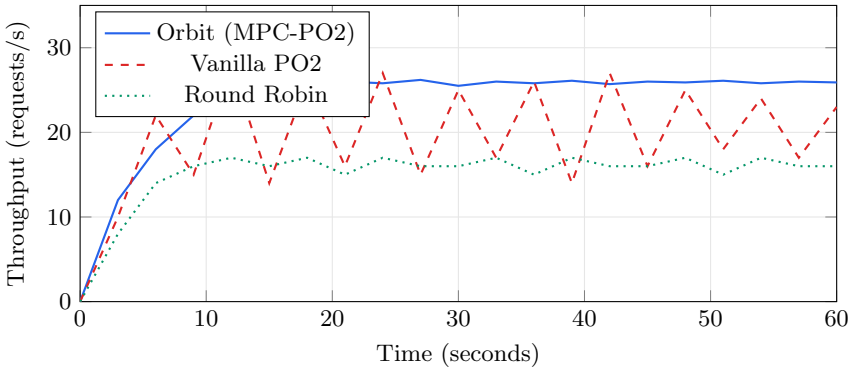

Fig. 2: Throughput over time. MPC-PO2 maintains stable throughput near 26 req/s, while vanilla PO2 oscillates between 14–28 req/s. Round Robin is stable but at lower throughput (16 req/s) due to poor load distribution.

4.4 Queue Dynamics Analysis

Figure 3 illustrates the mechanism behind MPC's stability. Vanilla PO2 allows queue depth to swing widely as it reacts to current state. MPC-PO2 maintains queue depth near the target $q^* = 5$ by predicting when the queue will grow and proactively reducing the server's weight. This predictive action prevents the extreme depths (up to 28 requests) that cause tail latency spikes.

4.5 Sensitivity Analysis

Regularization λ. Figure 4 shows latency as a function of λ. Very small λ leads to aggressive weight changes that can cause oscillation. Very large λ prevents meaningful adaptation. The optimal range $\lambda \in [0.25, 0.75]$ balances responsiveness with stability.

Horizon H. Performance improves from $H = 1$ to $H = 5$, then plateaus. Longer horizons provide diminishing returns because service rate predictions become less accurate. We use $H = 5$ as a practical default.

Controller Timestep Δt. Smaller Δt enables faster response but increases computational overhead. We find $\Delta t = 100$ms provides a good trade-off, with the solver completing well within this budget.

4.6 Scaling Behavior

Table 4 shows that Orbit scales well to larger configurations. Solve time grows linearly with server count but remains under 2ms even for 16 servers. Latency improvement decreases slightly at larger scales, as the law of large numbers provides some natural load balancing even without MPC.

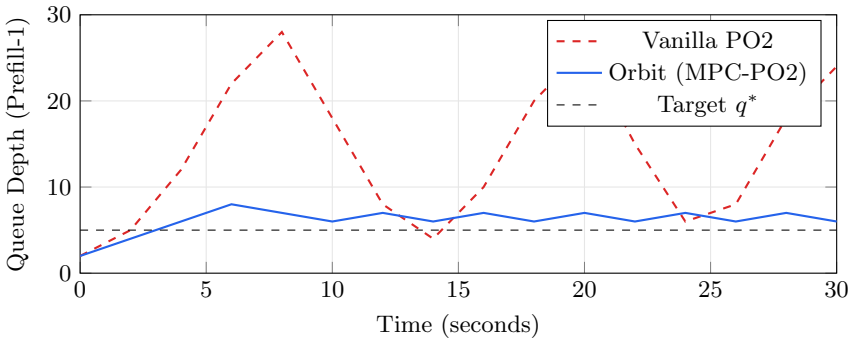

Fig. 3: Queue depth for the fast prefill server. Vanilla PO2 exhibits large swings (4–28 requests); MPC maintains depth near target $q^* = 5$ through predictive weight adjustment.

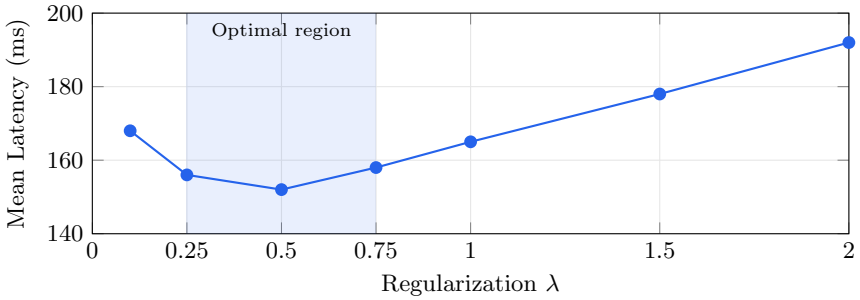

Fig. 4: Sensitivity to regularization parameter λ. Performance is robust for $\lambda \in [0.25, 0.75]$, with $\lambda = 0.5$ achieving the minimum.

5 Related Work

LLM Serving Systems. vLLM [1] introduced PagedAttention for efficient KV-cache management. Orca [2] proposed continuous batching. Splitwise [4] and DistServe [3] developed prefill-decode disaggregation. Sarathi-Serve [5] addresses stall-free scheduling. The vLLM router [6] provides production load balancing. These systems use standard algorithms (RR, PO2, consistent hashing); Orbit augments rather than replaces them.

Load Balancing Theory. The Power-of-Two-Choices [7] achieves exponential improvement in tail latency under idealized conditions. Join-Shortest-Queue [15] is optimal but requires global state. Consistent hashing [14] enables stateless routing with cache affinity. Recent work on learned load balancing [16] uses reinforcement learning but requires extensive training. Orbit uses model-based control, providing interpretability and immediate deployment without pre-training.

Table 4: Scaling to larger configurations

Config	Servers	Solve Time	Latency Improvement
2P2D	4	0.4 ms	+23.3%
4P4D	8	0.8 ms	+21.7%
8P8D	16	1.6 ms	+19.2%

Model Predictive Control. MPC has been applied to network congestion control [11], data center thermal management [12], and cloud resource allocation [13]. Williams et al. [9] developed sampling-based MPC for robotics. To our knowledge, Orbit is the first MPC application to LLM serving request routing.

AIOps and Intelligent Infrastructure. The emerging field of AIOps [17] applies machine learning to infrastructure management, including anomaly detection [18] and capacity planning [19]. Orbit contributes to this direction with a control-theoretic approach that provides guarantees absent in pure ML methods.

6 Discussion

Comparison with Cache-Aware Routing. The vLLM router's cache-aware policy optimizes for KV-cache hits by routing similar prompts to the same server. This is orthogonal to load balancing: cache-aware maximizes throughput via reuse, while MPC minimizes latency via load distribution. An interesting direction is combining them: use cache affinity to identify candidate servers, then apply MPC weights to choose among candidates.

Limitations. Our current evaluation uses simulation with simplified service time models. Key limitations include:

1. **Simplified dynamics:** Real service times depend on batch size, KV-cache state, and memory pressure in complex ways not captured by our model.
2. **No cache modeling:** We do not account for KV-cache hit rates, which significantly affect prefill latency.
3. **Simulation only:** Results on production vLLM deployments may differ.

Why MPC Over Reinforcement Learning? RL-based load balancing [16] can learn complex policies but requires: (i) extensive training, (ii) careful reward shaping, and (iii) re-training when conditions change. MPC offers: (i) immediate deployment with no training, (ii) interpretable optimization objectives, (iii) guaranteed constraint satisfaction, and (iv) automatic adaptation via online re-optimization. The trade-off is that MPC requires an explicit system model, which may be less accurate than a learned model for complex dynamics.

Future Work. We plan to: (i) validate on production vLLM deployments, (ii) extend the system model to incorporate KV-cache pressure and batching effects, (iii) explore hybrid approaches combining MPC with learned model components, and (iv) investigate integration with cache-aware routing.

7 Conclusion

We presented Orbit, a Model Predictive Control framework for adaptive request routing in disaggregated LLM serving. By formulating routing as an optimal control problem over queue dynamics, Orbit augments standard algorithms like Power-of-Two-Choices with predictive capability. Our simulation study demonstrates that MPC provides 15–16% mean latency reduction under steady load and 37% tail latency reduction under bursty conditions, with benefits concentrated in heterogeneous deployments where server capacities differ by 2–4×. Importantly, MPC adds no benefit (and slight overhead) for homogeneous deployments, providing clear deployment guidance: enable MPC when servers have different capacities, disable it otherwise. These results suggest that control-theoretic approaches offer a principled and practical path toward intelligent request scheduling in modern LLM serving systems.

Acknowledgments. We thank the anonymous reviewers for their feedback.

References

1. Kwon, W., et al.: Efficient memory management for large language model serving with PagedAttention. In: SOSP (2023)
2. Yu, G., et al.: Orca: A distributed serving system for transformer-based generative models. In: OSDI (2022)
3. Zhong, Y., et al.: DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In: OSDI (2024)
4. Patel, P., et al.: Splitwise: Efficient generative LLM inference using phase splitting. In: ISCA (2024)
5. Agrawal, A., et al.: Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In: OSDI (2024)
6. vLLM Project: vLLM Router: A high-performance load balancer for large-scale serving. https://github.com/vllm-project/router (2025)
7. Mitzenmacher, M.: The power of two choices in randomized load balancing. IEEE TPDS 12(10), 1094–1104 (2001)
8. Camacho, E.F., Bordons, C.: Model Predictive Control. Springer (2013)
9. Williams, G., et al.: Model predictive path integral control using covariance variable importance sampling. arXiv:1707.02342 (2017)
10. Garcia, C.E., Prett, D.M., Morari, M.: Model predictive control: Theory and practice. Automatica 25(3), 335–348 (1989)
11. Low, S.H., Lapsley, D.E.: Optimization flow control—I: Basic algorithm and convergence. IEEE/ACM ToN 7(6), 861–874 (1999)
12. Lazic, N., et al.: Data center cooling using model-predictive control. In: NeurIPS (2018)

13. Zhang, Y., et al.: Model predictive control for cloud resource management. In: ICDCS (2021)
14. Karger, D., et al.: Consistent hashing and random trees. In: STOC (1997)
15. Eschenfeldt, P., Gamarnik, D.: Join the shortest queue with many servers. Mathematics of OR 43(3), 867–886 (2018)
16. Mao, H., et al.: Learning scheduling algorithms for data processing clusters. In: SIGCOMM (2019)
17. Dang, Y., et al.: AIOps: Real-world challenges and research innovations. In: ICSE-SEIP (2019)
18. Zhao, N., et al.: Identifying bad software changes via multimodal anomaly detection for online service systems. In: FSE (2021)
19. Cortez, E., et al.: Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In: SOSP (2017)
20. Antsaklis, P.J., Michel, A.N.: Linear Systems. Birkhäuser (2006)